

Conecta 4 con Negamax

Documentación Técnica

Inteligencia Artificial

Índice

1. Descripción del Proyecto	2
2. Reglas del Juego	2
3. Representación del Estado	2
4. Algoritmo Negamax	2
4.1. Función de evaluación	2
4.2. Pseudocódigo Negamax con poda alfa-beta	3
4.3. Poda alfa-beta	3
5. Verificación de Victoria	3
6. Interfaz Gráfica	3
7. Flujo de Ejecución	4

1. Descripción del Proyecto

Implementación del juego **Conecta 4** con inteligencia artificial basada en **Negamax**. El jugador humano compite contra la CPU en un tablero de 6 filas por 7 columnas (configurable). El objetivo es alinear 4 fichas consecutivas en horizontal, vertical o diagonal antes que el oponente.

2. Reglas del Juego

- El tablero tiene 6 filas $\times$ 7 columnas (configurable hasta 10 $\times$ 10).
- Los jugadores alternan turnos colocando fichas en columnas.
- Las fichas caen por gravedad hasta la fila más baja disponible.
- Gana el primero en alinear 4 **fichas** consecutivas en cualquier dirección.
- Si el tablero se llena sin ganador, el juego termina en empate.

3. Representación del Estado

El tablero se almacena como una **matriz bidimensional** `int[filas][columnas]` donde:

- 0 = celda vacía
- 1 = ficha del jugador
- 2 = ficha de la CPU

La gravedad se modela buscando la fila más baja vacía de cada columna:

```
1 int filaDisponible(int col) {  
2     for (int f = filas-1; f >= 0; f--)  
3         if (tablero[f][col] == 0) return f;  
4     return -1; // columna llena  
5 }
```

4. Algoritmo Negamax

4.1. Función de evaluación

Negamax requiere evaluar tableros no terminales. La función `evaluar()` asigna puntuación analizando **ventanas de 4 celdas** en todas las direcciones:

- 4 fichas propias: +1000
- 3 fichas propias + 1 vacía: +50
- 2 fichas propias + 2 vacías: +10
- 3 fichas del oponente + 1 vacía: -80

4.2. Pseudocódigo Negamax con poda alfa-beta

```
1 función negamax(tablero, profundidad, alfa, beta, jugador):
2     si profundidad == 0 o juegoTerminado(tablero):
3         retornar evaluar(tablero, jugador)
4
5     mejorPuntuacion = -infinito
6     para cada columna disponible (0..6):
7         fila = filaDisponible(tablero, col)
8         tablero[fila][col] = jugador
9         puntuacion = -negamax(tablero, profundidad-1,
10                                -beta, -alfa, oponente)
11         tablero[fila][col] = 0
12         mejorPuntuacion = max(mejorPuntuacion, puntuacion)
13         alfa = max(alfa, puntuacion)
14         si alfa >= beta: break // poda
15     retornar mejorPuntuacion
```

4.3. Poda alfa-beta

La poda alfa-beta elimina ramas del árbol que no pueden influir en el resultado final, reduciendo la complejidad de $O(b^d)$ a $O(b^{d/2})$ en el mejor caso.

5. Verificación de Victoria

Se comprueban las 4 direcciones posibles para cada celda del tablero:

```
1 int[][] direcciones = {{0,1},{1,0},{1,1},{1,-1}};
2 // horizontal, vertical, diagonal /, diagonal \
3
4 boolean hay4Iguales(int fila, int col, int jugador) {
5     for (int[] dir : direcciones) {
6         int count = 0;
7         for (int k = 0; k < 4; k++) {
8             int f = fila + k*dir[0];
9             int c = col + k*dir[1];
10             if (dentroDelTablero(f,c) &&
11                 tablero[f][c] == jugador) count++;
12         }
13         if (count == 4) return true;
14     }
15     return false;
16 }
```

6. Interfaz Gráfica

La GUI fue construida con Java Swing:

- **Panel de tablero:** dibujado con `paintComponent()` usando círculos de colores.

- **Detección de columna:** al hacer clic, se calcula la columna por la posición X del mouse.
- **Animación:** la ficha se dibuja inmediatamente al colocarla con `repaint()`.
- **Selección de color:** diálogo inicial para que el jugador elija rojo o amarillo.

7. Flujo de Ejecución

1. El jugador selecciona color y nivel de dificultad.
2. El jugador hace clic en una columna; la ficha cae a la fila más baja.
3. Se verifica victoria o empate.
4. La CPU ejecuta Negamax y coloca su ficha en la columna óptima.
5. Se repite hasta estado terminal.